System Design

System Design

System design is the process of defining the architecture, components, modules, interfaces, and data flows of a software or hardware system to meet specified requirements.

Key Objective of system design

1. **Scalability**: Ensure the system can handle growth in users, data, and traffic.
2. **Reliability**: Guarantee that the system operates correctly, even in cases of hardware failures or unexpected errors.
3. **Performance**: Optimise response times, throughput, and resource utilisation.
4. **Maintainability**: Design for easy updates, debugging, and extensions.
5. **Security**: Safeguard data and operations from unauthorised access or breaches.

Types of System design

High-Level Design (HLD):

- Focuses on the overall architecture.
- Includes details about:
    - Components and their interactions.
    - Technology stack (databases, APIs, front-end, back-end).
    - Data flow and communication protocols.
- **Example**: Designing a microservices architecture for an e-commerce application.

Low-Level Design (LLD):

- Focuses on implementation details.
- Includes specifics such as:
    - Class diagrams.
    - Database schema.
    - API endpoint structures.
- **Example**: Defining database tables for user authentication.

Key Concepts in System Design

1. **Load Balancing**:
    - Distributes traffic across multiple servers to ensure even workload and prevent overloading.
2. **Caching**:
    - Stores frequently accessed data temporarily to reduce database load and improve response times.
    - Example: Using Redis or Memcached.
3. **Database Design**:
    - Deciding between relational (SQL) or non-relational (NoSQL) databases.
    - Optimising schema and indexing for performance.
4. **Sharding and Partitioning**:
    - Splitting databases or data into smaller, manageable parts for scalability.
5. **Message Queues**:
    - Enables asynchronous communication between system components.

    - Example: Using RabbitMQ, Kafka, or AWS SQS.
6. **CDN (Content Delivery Network)**:
    - Distributes static assets (e.g., images, CSS) to servers closer to users for faster delivery.
7. **Monitoring and Logging**:
    - Tracking system performance and errors for debugging and improvements.
8. **Fault Tolerance**:
    - Designing the system to handle and recover from failures without significant downtime.

Steps in System Design

1. **Requirement Gathering**:
    - Understand functional and non-functional requirements (e.g., scalability, performance).
2. **Architecture Design**:
    - Choose between monolithic, microservices, or serverless architectures.
3. **Component Design**:
    - Break down the system into individual components and define their roles.
4. **Database Design**:
    - Decide schema, normalisation, denormalization, and database technology.
5. **Communication Protocols**:
    - Define how components interact (e.g., REST, gRPC, WebSocket).
6. **Scalability and Reliability**:
    - Incorporate strategies like horizontal scaling, replication, and disaster recovery.
7. **Security**:
    - Implement measures like encryption, authentication, and authorization.

How to approach a system design?

first take requirement and divide three part

1. Function requirement
2. Non-functional requirement
3. Extended Requirement

then after evaluate user estimation & constraints per day/per second

How to improve the performance of an application or handle high traffic of an application?

1. Use caching
2. Sharding
3. Load balancing
4. Replication

5. Monitoring
6. Indexing on database level

What's the parameters for choosing horizontal designs and vertical designs?
Horizontal scaling excels in distributing workloads across multiple nodes. Vertical scaling suits scenarios where a single machine can handle the entire workload efficiently.

CAP

CAP

1. **Consistency (C)**
   - Every read receives the most recent write or an error.
   - All nodes in the system see the same data at the same time.
   - The system ensures that data is always in a consistent state across nodes.
   -

Example: If a bank transaction updates a balance on one server, all other servers immediately reflect the updated balance.

2. **Availability (A)**
   - Every request (read or write) receives a response, even if some nodes are down.
   - The system operates 24/7, ensuring high uptime and reliability.
   -

Example: A database is still accessible for reads and writes, even if some servers are unavailable.

3. **Partition Tolerance (P)**
   - The system continues to function even when there is a network partition (communication failure between nodes).
   - Distributed systems are expected to handle network failures without losing functionality.

Example: If nodes in a database cluster lose communication with each other due to a network issue, the system still serves requests, though with potential trade-offs in consistency or availability.

CAP Theorem in Practice

CA (Consistency + Availability):
- No tolerance for network partitions.
- Suitable for single-node or tightly coupled systems.
- **Example**: Traditional relational databases like PostgreSQL in a standalone setup.

CP (Consistency + Partition Tolerance):
- Sacrifices availability during network partitions.
- Ensures strong consistency even if some nodes are unreachable.
- **Example**: MongoDB in strong consistency mode.

AP (Availability + Partition Tolerance):
- Sacrifices strict consistency to ensure availability during partitions.
- Allows eventual consistency, where data across nodes becomes consistent over time.
- **Example**: Cassandra, DynamoDB.

Which CAP Theorem for NON-SQL like MongoDB vs Cassandra?

MongoDB:

It is generally classified as a CP (Consistency and Partition Tolerance) system.

Cassandra:

It is typically considered as an AP (Availability and Partition Tolerance) system.

MYSQL & ORACLE OR SQL:

Its generally prioritises Consistency and Availability (CA)

ACID

ACID

ACID transactions are a set of properties that ensure reliable processing of database transactions, especially in environments where multiple operations or users interact with a database.

Property	Meaning
Atomicity	All operations in a transaction either **complete fully or not at all**
Consistency	The database moves from one valid **state to another**
Isolation	Transactions **do not interfere** with each other
Durability	Once committed, data is **permanently saved**, even in case of failures

🔄 ACID in Action

🔸 1. Atomicity

If step 1 succeeds (₹1000 is debited), but step 2 fails (due to a system crash), the transaction is **rolled back**.

➡ No money is lost.
Either both debit and credit happen, or none do.

🔸 2. Consistency

Before the transaction: Total = ₹10,000
After transaction: Total must still = ₹10,000

➡ No data corruption.
The rules of the system (total balance) remain consistent.

🔸 3. Isolation

If another user is trying to check balances during the transfer, they won't see **half-updated** data.

➡ They either see the **before** or **after** state, not an in-between.
Transactions run as if they are the **only one** in the system.

🔸 4. Durability

Even if the power goes out right after the transfer is committed, the data is safe.

➡ It will not be lost.
Once you **commit**, it's **forever stored**.

Atomicity

- Ensures that a transaction is treated as a single, indivisible unit.
- Either **all operations** in the transaction are successfully completed, or **none** are applied.
- If any part of the transaction fails, the database is rolled back to its previous state.

Consistency

- Guarantees that a transaction transforms the database from one valid state to another valid state, adhering to all defined rules, constraints, and triggers.
- No data integrity violations occur.

Example: A transfer between accounts should ensure that the total balance remains unchanged:

- Before: Account A = 500, Account B = 300.
- After: Account A = 400, Account B = 400.

Isolation

- Ensures that the execution of a transaction is isolated from other transactions.
- Intermediate states of a transaction are not visible to other transactions.
- The outcome is as if transactions were executed sequentially, even if they run concurrently.

Example: Two transactions updating the same record must ensure one does not interfere with the other:

- Transaction 1 reads a balance of 500.
- Transaction 2 updates the balance to 600.
- Transaction 1 should either see the original value (500) or the final value (600), not an inconsistent intermediate state.

Durability

- Ensures that once a transaction is committed, the changes are permanent, even in the event of a system failure (e.g., crash or power outage).
- This is typically achieved through mechanisms like logging and backups.

Example: If a bank transfer is successfully committed, it will remain so even if the database server crashes immediately after.

Why ACID is Important

1. Prevents **data loss**
2. **Handles concurrent transactions safely**
3. **Ensures reliability in financial, inventory, and user-critical systems**

- Prevents issues such as:
    - **Dirty reads**: Reading uncommitted changes from another transaction.
    - **Lost updates**: Overwriting changes without considering concurrent operations.
    - **Inconsistent data**: Partial updates applied during transaction failures.

ACID:

1. **A - Atomicity**
    a. Ensures that each transaction is treated as a single unit, which either completes entirely or not at all. If any part of the transaction fails, the entire transaction is rolled back.
2. **C - Consistency**

   a. Ensures that a transaction brings the database from one valid state to another, maintaining database invariants
3. **I - Isolation**
   a. Ensures that transactions are executed in isolation from one another.
   b. Intermediate states of a transaction are invisible to other transactions.
4. **D - Durability**
   a. Ensures that once a transaction has been committed, it will remain so even in the event of a system failure

Consistency vs Availability
ACID prioritises consistency over availability, ensuring that all transactions are Processed reliably

Use Cases:
ACID is typically used in traditional relational databases where data integrity is crucial, such as banking systems.

BASE

BASE

It stands for **Basically Available, Soft state, Eventual consistency.**
BASE is used in distributed systems and NoSQL databases where high availability and scalability are more important, such as social media platforms.

These principles are often used in distributed systems and NoSQL databases.

Basically Available: The System guarantee availability, in terms of the CAP theorem, even in the presence of partial system failures

Soft state: The State of the system may change over time, even without input , due to eventual consistency.

Eventual Consistency: The system will become consistent over time, given that no new updates are made.

What is ACID vs BASE?

Design Approach

System Design Approach

Approach:

Low Level design-

1. What's the system design?
2. Requirement
    a. Functional requirements
    b. Non-functional requirements
    c. Extended requirements
    d. Estimation & Constraints
        i. Traffic
        ii.
3. Data Model design
4. What kind of database should we use?
5. API Design

High Level design-

1. Architecture
    a. Small-2 Micro-Services
2. What about inter-service communication and service discovery?
3. Detailed design

Reference:

https://dev.to/karanpratapsingh/system-design-content-delivery-network-cdn-bof

https://levelup.gitconnected.com/12-microservices-pattern-i-wish-i-knew-before-the-system-design-interview-5c35919f16a2

https://atul-agrawal.medium.com/web-hook-notification-platform-for-payment-transactions-e00ae678cd4d

What is Horizontal Scaling in design ?
Horizontal scaling or "scaling out" involves adding more machines to the resource pool.

What is vertical scaling in design ?
Vertical scaling or "scaling up" is based on adding power to an existing machine, such as RAM or a better CPU.

System Requirement

System Requirements:

1. Functional requirements:
These are requirements that define specific behaviour or functions, often represented by use cases.

How to Define: Start with user needs and business goals. Outline the primary tasks users should accomplish.
Working Backwards Approach: Begin with the end goal and trace steps backwards to identify requirements.
2. High Availability:
Availability is a measure of system uptime.

Time-based: E.g., 99.9% (or "three nines") uptime.
Count-based: E.g., 1 failure allowed in 1 million requests.
Design Principles: Redundancy, failover, replication, monitoring, and regular testing.
Processes: Regular maintenance, backup procedures, disaster recovery planning.
SLO vs SLA: Service Level Objective (SLO) is the target level of service. Service Level Agreement (SLA) is the contract with the customer promising certain SLOs.
3. Fault Tolerance, Resilience, Reliability:
Error: An incorrect internal state.
Fault: A component's incorrect behaviour.
Failure: When a system as a whole stops providing the required service.
Fault Tolerance: Ability of a system to behave in a well-defined manner once a fault occurs.
Resilience: Ability to return to a normal state after a failure.
Game Day vs Chaos Engineering: Game Day is a planned simulation. Chaos Engineering is introducing random failures.
Reliability: Probability a system will fail in a given period.
4. Scalability:
Vertical Scaling: Increasing the resources of a single node.
Horizontal Scaling: Adding more nodes.
Elasticity vs Scalability: Elasticity is the ability to scale automatically based on demand.
5. Performance:
Latency: Time taken to process a single operation.
Throughput: Number of operations processed in a given time period.
Percentiles: E.g., 95th percentile latency = under which 95% of the request latencies fall.
How to Increase: Optimize algorithms, caching, distributed systems.
6. Durability:
Ensuring data is safely stored and retrievable.

Backup Types:
Full: Everything is backed up.
Differential: Only changes since the last full backup.
Incremental: Only changes since the last backup of any type.
RAID: Redundant Array of Independent Disks. A method for redundant storage.
Replication: Copying data to ensure durability and availability.

Checksum: A computed value to detect errors in data.
7. Consistency:
Ensuring all users see the same data.

Eventual Consistency: Data will become consistent over time.
Linearizability: Real-time consistency.
Monotonic Reads: If a process reads the value of a data item, any successive reads by that process will always return that value or a more recent one.
Read-Your-Writes: Guarantees that a write by a process is visible to a subsequent read by the same process.
Consistent Prefix Reads: Guarantees that every read receives a prefix (some initial segment) of the writes, in order.
8. Maintainability, Security, Cost:
Maintainability: Ease with which a system can be maintained. Includes monitoring, testing, and deployment strategies.
Security:
CIA Triad: Confidentiality, Integrity, Availability.
Identity and Permissions Management: Who can do what.
Infrastructure Protection: Protecting physical and virtual resources.
Data Protection: Ensuring data confidentiality, integrity, and availability.
Cost Aspects: Engineering, maintenance, hardware, and software costs.
9. Summary of System Requirements:
Most Popular Non-Functional Requirements: Availability, Scalability, Performance, Security, Maintainability.
10. Regions, Availability Zones, etc.:
Regions: Geographic areas that consist of multiple, isolated data centers.
Availability Zones: Isolated locations within a region.
Data Centers: Physical facilities housing servers.
Racks: Structures holding multiple servers.
Servers: Physical machines running software.
11. Physical Servers to Serverless:
Physical Servers: Tangible hardware running software.
Virtual Machines (VMs): Emulated computer systems.
Containers: Lightweight VMs that share the same OS kernel and isolate the application processes from each other.
Pros: Lightweight, fast, consistent environment, scalable.
Cons: Less insolation than VMs.
Serverless: Cloud-computing model which can run individual functions in response to events. Resources are fully managed by the cloud provider.
Pros: Scalability, no infrastructure management, cost-effective.
Cons: Cold starts, limited customization, statelessness.
12. Synchronous vs. Asynchronous Communication:
Synchronous: Sender waits for the receiver to respond.
Asynchronous: Sender sends the message and proceeds without waiting for a response.
13. Asynchronous Messaging Patterns:
Message Queuing: Temporarily storing messages to be processed later.
Publish/Subscribe: Senders (publishers) send messages to channels without specifying receivers. Receivers (subscribers) subscribe to channels they're interested in.

Competing Consumers: Multiple consumers process messages from a single channel, distributing load.
Request/Response Messaging: A two-way messaging pattern; a request is followed by a response.
Priority Queue: Messages are processed based on priority rather than order of arrival.
Claim Check: Large message is dropped off at a service and a claim check is provided to the requester, which can be used to get the full message later.

14. Network Protocols:

TCP (Transmission Control Protocol): Reliable, connection-oriented protocol.
UDP (User Datagram Protocol): Connectionless, no guarantee of message delivery.
HTTP (Hypertext Transfer Protocol): Application protocol for distributed, collaborative, hypermedia systems.
HTTP Request and Response: Basic units of HTTP communication, where a client sends a request and a server responds.

15. Blocking vs. Non-blocking I/O:

Blocking I/O: The process is blocked until data is available.
Non-blocking I/O: The process continues even if the data is not available.
Thread per Connection Model: Each connection gets a dedicated thread.
Thread per Request with Non-blocking I/O Model: Threads are used only when processing a request.
Event Loop Model: Single thread handles multiple connections using events.
Concurrency vs. Parallelism: Concurrency is multiple tasks starting, running, and completing in overlapping time periods, while parallelism is multiple tasks running at exactly the same time.

16. Data Encoding Formats:

Textual Formats (e.g., JSON, XML): Human-readable, more overhead.
Binary Formats (e.g., Protobuf, Avro): Efficient, not human-readable.
Schema Sharing Options: How schema (structure) of data is shared between sender and receiver.
Backward and Forward Compatibility: Ensuring newer systems can read older data and older systems can read newer data, respectively.

17. Message Acknowledgment:

Safe Acknowledgment Modes: Receiver confirms safe receipt of a message.
Unsafe Acknowledgment Modes: Sender doesn't wait for a confirmation.

18. Deduplication Cache:

Local vs. External Cache: Storage close to the CPU vs. external storage systems.
Adding Data to Cache: Can be added explicitly by the application or implicitly by the caching system.
Cache Data Eviction: Removing data from cache, can be based on size, time, or other criteria.
Expiration vs. Refresh: Time after which data is removed vs. updating data periodically.

19. Metadata Cache:

Cache-aside Pattern: Application is responsible for loading cache.
Read-through and Write-through Patterns: Cache controls the data load and update.
Write-behind (Write-back) Pattern: Data is written to cache and is asynchronously written back to the storage.

20. Queue:

Bounded and Unbounded Queues: Limited vs unlimited size.

Circular Buffer (Ring Buffer): Fixed-size data structure that uses a single, continuous region of memory.

21. Full and Empty Queue Problems:

Load Shedding: Discarding excess traffic.
Rate Limiting: Limiting the rate of requests by users.
Failed Requests Handling: Strategies like retries or moving to dead-letter queues.
Backpressure: Signal to the producer to slow down.
Elastic Scaling: Dynamically adjusting resources based on demand.

22. Start with Something Simple:

Single Machine vs. Distributed System Concepts: Many concepts in distributed systems (like caching, load balancing) can be understood with the analogy of single-machine concepts.
Interview Tip: For system design interviews, start with a basic solution and then incrementally add complexity based on the requirements and constraints presented.

23. Blocking Queue and Producer-Consumer Pattern:

Producer-Consumer Pattern: Separate components that produce and consume data, often operating at different rates.
Wait and Notify: Mechanisms to synchronize between producers and consumers.
Semaphores: A synchronization primitive used to control access to a shared resource.
Blocking Queue Applications: Used in thread pooling, task scheduling, etc.

24. Thread Pool:

Pros: Efficient use of resources, controlled resource usage, quick task start-up.
Cons: Complexity, potential resource contention.
CPU-bound vs. I/O-bound tasks: CPU-bound tasks spend most of their time using the CPU, while I/O-bound tasks spend time waiting for external operations (like disk or network operations).
Graceful Shutdown: Properly terminating threads and releasing resources when shutting down a pool.

25. Big Compute Architecture:

Batch Computing Model: Processing high volumes of data where a group of transactions is collected over a period.
Embarrassingly Parallel Problems: Problems where little or no effort is needed to separate the problem into tasks to run in parallel.

26. Log:

Memory vs. Disk: Storing logs in volatile memory (faster, but not persistent) vs. non-volatile disk storage (slower but persistent).
Log Segmentation: Splitting logs into segments to manage them more efficiently.
Message Position (Offset): The position of a message within a log.

27. Index:

How to Implement an Efficient Index for a Messaging System: Use data structures like B-trees or Hash Indexes to allow for quick lookups and writes.

28. Time Series Data:

Storing and retrieving data points indexed in time order.

How to Store and Retrieve: Use databases optimized for time series data, like InfluxDB, to ensure efficient storage and fast queries.

29. Simple Key-Value Database:

How to Build: Utilize hash tables for quick lookups. Address issues like collision resolution.
Log Compaction: Reducing the size of logs by removing redundant data.

30. B-tree Index:
Usage in Databases and Messaging Systems: B-trees allow for efficient data storage and retrieval in databases and are used to index data in messaging systems.
31. Embedded vs. Remote Database:
Embedded Database: Runs within the same address space as the application. E.g., SQLite.
Remote Database: Separate from the application and is accessed over a network. E.g., PostgreSQL, MySQL.
32. RocksDB:
A high-performance embedded database for key-value data.

Memtable: In-memory data structure where RocksDB writes are inserted.
Write-ahead Log (WAL): A log that stores changes to data, ensuring durability.
Sorted Strings Table (SSTable): Persistent, ordered immutable map of key-value pairs.
33. LSM-tree vs. B-tree:
Log-Structured Merge-Tree (LSM-tree): Optimized for systems that write data more often than reading.
Write Amplification vs. Read Amplification: Trade-offs between the cost of writing data vs. reading data.
34. Page Cache:
Increasing Disk Throughput: Techniques like batching and zero-copy read can help in efficiently reading/writing to disks.
35. Push vs. Pull:
Push: Server initiates sending data to the client.
Pull: Client requests data from the server.
36. Host Discovery:
DNS: The Domain Name System translates domain names to IP addresses.
Anycast: Routing strategy where a single destination address has multiple routing paths to two or more endpoint destinations.
37. Service Discovery:
Server-side and Client-side Discovery Patterns: Mechanisms where services find each other's network locations.
Service Registry: A database containing the network locations of service instances.
38. Peer Discovery:
Peer Discovery Options: How nodes in a network find each other.
Gossip Protocol: Nodes randomly exchange information, which gradually propagates through the system.
39. Choosing a Network Protocol:
TCP, UDP, and HTTP: Decision depends on use case requirements like reliability, speed, and connection state.
40. Video over HTTP:
Adaptive Streaming: Adjusting video quality in real-time based on the viewer's network and playback conditions.
41. CDN (Content Delivery Network):
How to Use & Benefits: CDNs distribute content across multiple servers to optimize access for users globally. They reduce latency and protectagainst traffic surges.

Point of Presence (POP): Physical locations or access points that connect to the internet and host servers, acting as the local source for cached content.

42. Push and Pull Technologies:
Short Polling: Client frequently asks the server for new data.
Long Polling: Client requests information and waits for the server to respond.
Websocket: Protocol that allows two-way communication with a server over a single, long-lived connection.
Server-Sent Events: Server pushes updates to the client over an HTTP connection.
43. Push and Pull Technologies in Real-Life Systems:
Quiz: Real-life examples would be chat applications (like Slack) using Websockets for real-time messaging, or stock trading apps using long polling for updating stock prices.
44. Large-scale Push Architectures:
C10K and C10M Problems: Challenges of handling 10,000 and 10 million concurrent connections respectively.
Large-Scale Push Architectures: Techniques like event-driven programming to handle large numbers of simultaneous connections.
Problems of Long-Lived Connections: Resource management, detecting stale or "dead" connections, and handling dropped connections.
45. Building Reliable, Scalable, and Fast Systems:
Common Problems in Distributed Systems: Network failures, machine failures, performance bottlenecks.
System Design Concepts: Caching, load balancing, sharding, replication, and partitioning.
Three-Tier Architecture: Presentation, logic, and data tiers.
46. Timeouts:
Fast Failures vs. Slow Failures: Quickly failing a request vs. taking a long time before failing.
Connection and Request Timeouts: Time limits set for establishing a connection or waiting for a response.
47. Handling Failed Requests:
Strategies:
Cancel: Terminate the request.
Retry: Attempt the request again.
Failover: Redirect the request to another system or component.
Fallback: Use an alternate backup solution.
48. When to Retry:
Idempotency: Ensuring that operations can be repeated without side effects.
AWS API Failures: Not all API failures should be retried. For instance, validation errors shouldn't be retried.
49. How to Retry:
Exponential Backoff: Increase the wait time between retries exponentially.
Jitter: Adding randomness to retry intervals to spread out the load.
50. Message Delivery Guarantees:
At-most-once: Messages are delivered once or not at all.
At-least-once: Messages are delivered one or more times.
Exactly-once: Messages are delivered precisely once.
51. Consumer Offsets:
Log-Based Messaging Systems: Systems where messages are stored in order and consumers track their position using offsets.
Checkpointing: Periodically saving the state of a process.
52. Batching:

Pros and Cons: Batching can improve efficiency but may introduce latency.
Handling Batch Requests: Ensuring that all items in a batch are processed, even if some fail.

53. Compression:

Pros and Cons: Reduces data size but adds computational overhead.
Compression Algorithms: Techniques like GZIP, Brotli, and LZ4, each with their trade-offs.

54. Scaling Message Consumption:

Single Consumer vs. Multiple Consumers: One consumer can ensure order but is limited in throughput. Multiple consumers can increase throughput but introduce complexity in message ordering.
Problems with Multiple Consumers: Ensuring order, handling duplicate processing, etc.

55. Partitioning in Real-Life Systems:

Pros and Cons: Helps in horizontal scaling and distributing loads but can introduce complexities.
Applications: Databases, messaging systems, etc.

56. Partitioning Strategies:

Lookup Strategy: Direct lookup.
Range Strategy: Based on a range of data values.
Hash Strategy: Using a hash function for uniform distribution.

57. Request Routing:

Physical and Virtual Shards: Actual data partitions vs logical partitions.
Routing Options: Methods for directing a request to the appropriate server or data partition.

58. Rebalancing Partitions:

Ensuring data is evenly distributed across systems, especially when adding or removing nodes.

59. Consistent Hashing:

Implementation: Uses a circular keyspace.
Advantages and Disadvantages: Reduces the number of re-mapped keys when scaling but can lead to non-uniform data distribution.
Virtual Nodes: Multiple hash values for a single node to ensure a more uniform distribution.

60. System Overload:

Importance: Protecting against system overload ensures continued availability and prevents cascading failures across dependent systems.
Protection Mechanisms: Load shedding, rate limiting, backpressure, and queuing.

61. Autoscaling:

Scaling Policies:
Metric-based: Scale based on metrics like CPU utilization or memory usage.
Schedule-based: Scale based on known usage patterns or times.
Predictive: Scale based on predicted future traffic patterns.
Autoscaling System Design: Involves metrics collection, decision-making algorithms, and mechanisms to add or remove resources.

62. Load Shedding:

Implementation: Prioritize and process only the most critical tasks, while dropping or delaying non-essential tasks.
Considerations: Decide which requests to drop, how to notify clients, and when to trigger load shedding.

63. Rate Limiting:

Purpose: To prevent any single user or client from overloading the system.

Implementation: Token buckets, leaky buckets, and fixed window counters are popular methods.

64. **Synchronous and Asynchronous Clients:**
Admission Control Systems: Mechanisms to decide whether to process a request immediately or defer it.
Blocking I/O vs. Non-Blocking I/O Clients: Synchronous vs. Asynchronous client operations.

65. **Circuit Breaker:**
Finite-State Machine: Mechanism where the circuit breaker maintains states (Closed, Open, Half-Open) to decide whether to process or deny requests.
Considerations: Setting thresholds, reset intervals, and monitoring the health of downstream services.

66. **Fail-Fast Design Principle:**
Problems with Slow Services: Slow services can cause chain reactions leading to larger system outages.
Solutions: Implementing timeouts, circuit breakers, and other mechanisms to quickly fail and return control to the caller.

67. **Bulkhead**:
Implementation: Isolating parts of a system so that if one fails, it doesn't bring down the whole system.
Applications: Can be applied at various levels, from processes to servers to data centers.

68. **Shuffle Sharding:**
Implementation: Combining sharding and redundancy to reduce the blast radius of failures.
Applications: Reducing the impact of failures in multi-tenant systems.

69. **Host Discovery:**
DNS: Translates domain names to IP addresses, enabling clients to discover hosts.
Anycast: Uses DNS to route to the nearest or best-performing physical location.

70. **Service Discovery:**
Server-side and Client-side Discovery Patterns: Mechanisms where services find each other's network locations, either with a central server or clients querying a service registry.
Service Registry: Stores locations of service instances, allowing for service discovery.

71. **Peer Discovery:**
Peer Discovery Options: Mechanisms like bootstrap lists or centralized directories.
Gossip Protocol: Nodes share information about their peers, allowing for decentralized discovery.

72. **Choosing a Network Protocol:**
TCP vs. UDP vs. HTTP: Depending on the requirements (reliability, connection setup, data size), one might be chosen over the other.

73. **Network Protocols in Real-Life Systems:**

Analyzing different scenarios and challenges to choose the best protocol. For example, streaming services might prefer UDP, while banking transactions would use TCP.

74. **Video Over HTTP:**
Adaptive Streaming: The video quality is dynamically adjusted based on the viewer's network conditions.

75. **CDN**:
Benefits: Faster content delivery by caching content closer to the end-users and reducing origin server load.
How It Works: Content is distributed and stored across a network of servers. When a user requests content, it's served from the nearest cached server.

76. **Push and Pull Technologies in Real-Life Systems:**
Real-world applications like social media updates or live sports updates may use push technologies like WebSockets, while email clients might use pull technologies to check for new mails.

77. **Large-Scale Push Architectures:**
Design considerations for building systems that can handle millions of simultaneous connections.

78. **Timeouts**:
Importance: Ensure that a system doesn't hang indefinitely.
Types:
Connection Timeouts: Time a task will wait while attempting to establish a connection.
Read/Write Timeouts: Time a task will wait for a read or write operation to complete.
Handling: Opt for graceful degradation of service, provide feedback to users, and use retries judiciously.

79. **Handling Failed Requests:**
Logging: Essential for diagnostics. What failed, where, and why?
Retry Logic: Implementing algorithms like exponential backoff.
Notifications: Inform stakeholders or trigger systems about persistent failures.

80. **When to Retry:**
Transient vs Permanent Failures: Temporary glitches (network blip) vs. consistent failures (invalid credentials).
Safety: Ensuring retries don't exacerbate the problem.

81. **How to Retry:**
Immediate vs Delayed: Waiting between retry attempts can be crucial.
Exponential Backoff: Increasingly long waits between retries.
Jitter: Introducing randomness to retry intervals.

82. **Message Delivery Guarantees:**
Importance: Ensuring data isn't lost or seen multiple times.
Strategies: Acknowledgments, transaction logs, checkpoints.

83. **Consumer Offsets:**
Checkpointing: Storing position in a stream or log.
Benefits: Allows consumers to pick up where they left off after failures.

84. **Batching**:
Benefits: Can significantly reduce overheads and increase throughput.
Challenges: Introducing latency, complexity in error handling.

85. **Compression**:
Trade-offs: CPU time vs. bandwidth.
Use Cases: Important in bandwidth-constrained environments or for storage savings.

86. **Scaling Message Consumption**:
Parallelism: Distributing load across multiple consumers.
Ordering: Ensuring that the sequence is maintained, if necessary.

87. **Partitioning in Real-Life Systems:**
Consistency vs Availability: Deciding trade-offs based on system requirements.

88. **Partitioning Strategies:**
Consistency Hashing: Minimizing reorganization of data when nodes are added or removed.

89. **Request Routing:**
Load Balancers: Distributing incoming requests to prevent any single resource from being overwhelmed.
Content-based Routing: Directing traffic based on the content of the request.

90. **Rebalancing Partitions:**
Importance: Ensuring data and load are evenly distributed across nodes.
Strategies: Manual interventions, algorithmic rebalancing.

91. **Consistent Hashing:**
Virtual Nodes: A mechanism to ensure a more uniform distribution of data.

92. **System Overload:**
Monitoring and Alerts: Real-time insights into system health.
Throttling: Limiting user or service requests.

93. **Autoscaling**:
Cloud Services: Many cloud providers offer autoscaling services that can automatically add or remove resources based on rules or machine learning.

94. **Load Shedding:**
Dynamic Adjustments: Dropping non-critical tasks when the system is under heavy load.

95. **Rate Limiting:**
API Gateways: Many come with built-in rate limiting features.

Client Feedback: Informing clients when they're exceeding limits.

96. **Synchronous and Asynchronous Clients:**
Trade-offs: Responsiveness vs. resource usage.
Use Cases: Asynchronous operations for long-running tasks.

97. **Circuit Breaker:**
Resilience Patterns: Preventing system failures from cascading.

98. **Fail-Fast Design Principle:**
Health Checks: Regular checks to ensure services are operational.
Fallbacks: Having backup systems or data sources in place.

99. **Bulkhead**:
Isolation: Making sure failures in one part don't impact others.

100. **Shuffle Sharding:**
Resilience: Improving system reliability by reducing the impact radius of failures.

SD Imp Questions

System design

Design LRU cache

LRU cache stands for Least Recently Used cache.
The size of the cache is fixed and it supports get () and put () operations
When the cache is full, the put () operation removes the least recently used cache.

Design a data structure for LRU cache.

It should support the following operations
Get and set.

If you are using a design pattern then how can you use design?

Design and Write Retry mechanism until transaction success and doesn't throw any kind of exception

elevator system and pub/sub system

Design instagram.

https://techwithkp.com/high-level-design-interview-how-to-approach-it-and-ace-it/

https://leetcode.com/discuss/interview-question/system-design/6149386/System-Design-or-Design-a-URL-Shortener-(Tiny-URL)-or-Interview-Question

Reference

Reference

https://github.com/karanpratapsingh/system-design?tab=readme-ov-file

https://interviewready.io/blog/system-design-of-whatsapp-calling-app

https://leetcode.com/problems/frequency-of-the-most-frequent-element/solutions/1175088/C++-Maximum-Sliding-Window-Cheatsheet-Template/

https://talent500.co/blog/

DRY principle

DRY Principle

https://www.geeksforgeeks.org/dont-repeat-yourselfdry-in-software-development/

The Don't Repeat Yourself (DRY) principle is a fundamental concept in software development that aims to reduce redundancy and improve maintainability by ensuring that every piece of knowledge or logic exists in only one place within a system.

This principle, formulated by Andy Hunt and Dave Thomas in their book The Pragmatic Programmer, emphasizes the importance of avoiding duplicated code and processes.

Software Architecture

Scalability

Scalability Formal

Traffic/Load may increase:

1. Over time
2. On the Seasonal pattern
3. Sporadically

Definition: The Measure of a system ability to handle a growing amount of work, in an easy and cost effective way, by adding resources to the system.

Types of Scalability (Three Orthogonal way –)

1. **Scale Up/Vertical Scalability**:
    a. **Definition**: Adding more resources or upgrading the existing resources on a single computer, to allow our system to handle higher traffic or load.
    b. **Pros**:
        i. Any application can benefits from it
        ii. No code changes are required
        iii. The Migration between different machines is very easy
    c. **Cons**:
        i. The Scope of upgrade is limited
        ii. We are locked to a centralised system which cannot provide
            1. High Availability
            2. Fault Tolerance
        iii.
2. **Scale out/Horizontal Scalability**:
    a. **Definition**: Adding more resources in the form of new instances running on different machines, to allow our system to handle higher traffic or load.
    b. **Pros**:
        i. No Limit on scalability
        ii. It's easy to add/remove machines
        iii. If designed correctly we get
            1. High Availability
            2. Fault Tolerance
    c. **Cons**:
        i. Initial code changes may be required
        ii. Increased complexity, coordination challenges

3. **Team/Organization Scalability**: (this is last not least)
    a. **Definition**: The measure of a system's ability to handle a growing amount of work (features, testing, fixing bugs, release), in an easy and cost effective way, by adding resources to the system.
    b. OR - Increasing productivity while hiring more engineers into the team.

Reasons for Productivity Degradation

1. Many crowded meetings
2. Code merge conflict
3. Business complexity - Longer ramp up time
4. Testing is harder and slower
5. Release become very risky

Software Architecture impacts engineering velocity (team productivity)

Availability

Availability

Definition:
The Fraction of time/probability that our service is operationally functional and accessible to the user.

1. Availability is an important attribute when designing a system
2. It has the greatest impact on:
    a. Our users
    b. Our business

Importance and Risk of Availability - Users

1. Users try to purchase from online store but:
    a. The page doesn't load
    b. They get an error at checkout
2. Users lose access to their email for considerable time
    a.

Importance of availability - Business (B2B)

1. If we provide services to other companies, the impact of outage is compounded
2. AWS simple storage service had an outage in feb 2017
3. Since many companies and websites used the service, it almost took down the entire internet.

Importance of Availability - Mission Critical services

1. System downtime is not always just an inconvenience
2. Our software can be used for mission-critical service like
    a. Air traffic control in airports
    b. Healthcare in hospitals
3. If it crashes/becomes unavailable for prolonged periods, people's lives are on line.

Impact of downtime on the business

1. For a business, the consequences of our system going down are:
    a. If users cannot use our system, our profits become zero
    b. If users lose access constantly, they go to our competitors
2. Thus, downtime of our system for a business results in a loss of
    a. Money
    b. Customers

Alternative way to calculate/estimate availability

1. MTBF
2. MTTR

MTBF:- Mean Time Between failures

1. Represents the average time our system is operational
2. Useful when:
    a. Dealing with multiple piece of hardware
    b. Each component has its own operational shelf life
    c. We are not using cloud/third party API with a given uptime and availability

MTTR: - Mean Time to recovery

1. Time average it takes us to detect and recover from a failure
2. Average downtime of our system

Availability - MTBF/(MTBF + MTTR)

How much is high availability?

1. Users would want 100% availability
2. On the engineering side it is:
    a. Extremely hard to achieve
    b. Leaves no time for maintenance/upgrades

Industry Standard of Availability

1. No Strict definition of what constitutes high availability
2. Industry standard set by cloud vendors is typically 99.9% or higher

System availability - uptime / (uptime + downtime)

Fault Tolerance & High Availability

Fault Tolerance & High Availability

1. What prevents us from achieving High Availability - sources of failure
2. Methods of Achieving High availability & fault tolerance

There are three categories of error

1. **Human error**
    a. Pushing a faulty config to production
    b. Running the wrong command/script
    c. Deploying an incompletely tested new version of software
2. **Software error**
    a. Long garbage collections
    b. Out-of-memory exceptions
    c. Null pointer exceptions
    d. Segmentation faults
3. **Hardware failure**
    a. Servers/routers/storage devices breaking down due to limited shelf-life
    b. Power outages due to natural disasters
    c. Network failures because of:
        i. Infrastructure issues
        ii. General congestion

Introduction to fault tolerance

1. Failures will happen despite
    a. Improvements to our code
    b. Review, testing and release processes
    c. Performing ongoing maintenances to our hardware
2. Fault tolerance is the best way to achieve High availability in our system

Definition: Fault tolerance enables our system to remain operational and available to the users despite failures within one or multiple of its components.

Tactics for achieving fault tolerance

1. Fault tolerance revolves around 3 major tactics:
    a. Failure Prevention
    b. Failure detection and Isolation
    c. Recovery

Failure Prevention:

1. To prevent our entire system from going down, eliminate any single point of failure in our system
2. Examples of a single point of failure can be:
    a. One server where we are running our application
    b. Storing all our data on the one instance of our database that runs on a single computer
3. Best way to eliminate a single point of failure is through replication and redundancy

Types of Redundancy

1. **Spatial Redundancy**: Running replicas of our application on different computers
2. **Time Redundancy**: Repeating the same operation/request multiple times until we succeed/give up

Strategies for redundancy and replication

We have two strategies which are extensively used in the industry in different systems

1. Active-Active Architecture
    a. In this architecture, all database will be sync with same level
2. Active-Passive Architecture
    a. Active passive architecture is basically one primary replica and rest all passive replica and passive replica follow to primary replica

Advantages of Active-Active Architecture

1. Load is spread among all the replica
2. Identical to horizontal scalability
3. Allows more traffic
4. Better performance

Disadvantages of Active-Active Architecture

1. All the replicas are taking requests
2. Additional coordination required to keep active replicas in sync

Disadvantages of Active-Passive Architecture

1. Ability to scale our system is lost
2. All the requests still go to only one machine

Advantages of Active-Passive Architecture

1. Implementation is easier
2. There is a clear leader with up-to-date data
3. Rest of the replica are followers

Failure Detection

Monitoring services are checking health for each instances if any server is no longer available then will notify

False Positive

1. But the issues might be the network or long garbage collection
2. Then the monitoring service is going to have a false positive
3. It assumed that a healthy host is faulty
4. Monitoring work for false positive not for false negative

False Negative

1. The monitoring service shouldn't have false negatives
2. False negative mean that:
    a. The server may have crashed
    b. The monitoring system did not detect that

Monitoring system functions

1. Exchange of messages in the form of pings and heartbeats
2. Collect data about the number of errors each host gets per minute
    a. If the error rate in one of the hosts is high, it can interpret that as failure of the host
3. Collect information about time taken for each host to respond
    a. If the time to respond to requests becomes long, it can decide that the host is slow

Recovery from failure (fast recovery = 0)

- Availability = MTBF / (MTBF + MTTR)
- Actions after detecting faulty instance/server:
    - Stop sending traffic/workload to that host
    - Restart the host to make the problem go away
    - Rollback - going back to a version that was stable and correct

Rollbacks

1. Common in databases
2. If we get to a state violating some condition/data, we can roll back to the last correct state in the past
3. If we detect errors while rolling out new versions of software, we can roll back to the previous version

SLA, SLO, SLI

SLA, SLO, SLI

SLA - Service level Agreement

- It is a legal contract that represents our quality service such as:
    - Availability
    - Performance
    - Data durability
    - Time to respond to system failures
- It states the penalties and financial consequences, if we breach the contract
- The penalties include:
    - Full/Partial refunds
    - Subscription/License extensions
    - Service credits

SLA and Users

- SLA exist for:
    - External paying users (always)
    - Free External users (sometimes)
    - Internal users within our company (occasionally)
- Internal SLA don't include any penalties
- SLA for free external users makes sense if our system has major issues during a free trial of our service
- We compensate those users with a trial extension or credits for future
- Companies providing entirely free services don't publish SLA